The periodic-equilibrium solver

Where it came from, exactly how it computes, how we know it is right, and porting it to C++

R. P. Gregorio • Kasai Laboratory, Institute of Science Tokyo

This document is a complete defence of *one* piece of the pipeline: the routine that drives the regolith to its periodic steady state, on which the whole K_d retrieval rests. It covers (1) how the method came about, (2) the old and new calculations in full numerical detail, (3) precisely how they differ, (4) how we know the new one is right, and (5) a concrete C++ port. Animated companions: `spinup.gif`, `newmethod.gif`, `reconstruct.gif` (in this folder).

1 How the method came about

1.1 What has to be solved

To retrieve K_d we compare modelled deep temperatures to the Apollo sensors. "Modelled" means: integrate the 1-D heat equation in the regolith,

$$\rho(z)\, c_p(T)\, \partial_t T = \partial_z\big(K(T,z)\, \partial_z T\big), \tag{1}$$

under a periodic Sun until the column reaches its *periodic steady state* — the condition where each lunation is identical to the last. Only then can we read off the cycle-mean temperature $\langle T\rangle(z)$ at the sensor depths and compare. Everything hinges on actually reaching that state.

1.2 The naive method, and the bug we hit (flag F1)

The original code reached steady state the obvious way: start from a guessed profile and step Eq. (1) forward for a fixed number of lunations (about 30), declaring success when the cycle-to-cycle change fell below 0.01 K. **The symptom that something was wrong:** the retrieved K_d moved when we changed the *starting* temperature — a physical answer must not depend on an arbitrary initial guess.

The diagnosis: a fixed 30-lunation spin-up never reaches steady state at sensor depth. The column relaxes on the diffusive timescale of its full depth,

$$\tau \sim \frac{z_{\max}^2}{\kappa} \approx \text{order } 10^3 \text{ lunations} \qquad (z_{\max} = 5\,\text{m},\ \kappa \sim 10^{-8}\,\text{m}^2\text{s}^{-1}), \tag{2}$$

so after 30 lunations the deep regolith still holds its initial value, and that value leaks straight into K_d. Worse, the convergence test *lied*: because the deep drift is so slow, the cycle-to-cycle change is already < 0.01 K while the profile is still far from settled. The model looked converged when it was not. (This is documented as audit flag F1 in `docs/FLAG_REPORT.md`.)

1.3 The insight that fixed it

The escape came from asking what the steady state actually *is*, rather than waiting for it. Below the diurnal skin there are no heat sources, so at steady state the same geothermal flux Q_b passes through every layer; with Fourier's law $q = K\,\partial_z T$ this fixes the deep gradient exactly:

$$K(\langle T\rangle, z)\,\frac{d\langle T\rangle}{dz} = Q_b \quad\Longrightarrow\quad \frac{d\langle T\rangle}{dz} = \frac{Q_b}{K(\langle T\rangle, z)}. \tag{3}$$

The deep profile is therefore *not* something to be discovered by long integration — it is determined by Eq. (3) once the temperature at a single shallow "anchor" depth is known. That observation is the whole method: settle the fast skin to get the anchor, then *impose* Eq. (3) for the deep column instead of waiting for diffusion to find it. The result is the flux-anchored solver, `lunar/equilibrium.py`.

2 The old calculation, in full detail

2.1 Discretisation

The column is a geometric grid of $N \approx 69$ cells, millimetre-thin at the surface and growing to 5 m. Time advances in $\Delta t = 3600$ s steps (~709 steps per lunation). Equation (1) is integrated with Crank–Nicolson (second-order, unconditionally stable), with material properties frozen at the explicit half-step (the standard linearisation for a mildly non-linear diffusion).

2.2 One time step

For interior cell i define the face conductances $\alpha_i^L = \Delta t\, K_{i-1/2}/(\delta z_i\, c_i)$ and $\alpha_i^R = \Delta t\, K_{i+1/2}/(\delta z_{i+1}\, c_i)$, where $K_{i\pm1/2}$ are *harmonic-mean* face conductivities, $c_i = \rho_i c_{p,i} \Delta z_i$ is the heat capacity of the cell, and δz are centre-to-centre spacings. One Crank–Nicolson step solves the tridiagonal system $A\,T^{n+1} = d$ with

$$a_i = -\tfrac{1}{2}\alpha_i^L, \quad b_i = 1 + \tfrac{1}{2}(\alpha_i^L + \alpha_i^R), \quad c_i = -\tfrac{1}{2}\alpha_i^R, \tag{4}$$

$$d_i = \tfrac{1}{2}\alpha_i^L\, T_{i-1}^n + \left[1 - \tfrac{1}{2}(\alpha_i^L + \alpha_i^R)\right] T_i^n + \tfrac{1}{2}\alpha_i^R\, T_{i+1}^n. \tag{5}$$

This is solved by the **Thomas algorithm** (one forward sweep, $c'_i = c_i/(b_i - a_i c'_{i-1})$, $d'_i = (d_i - a_i d'_{i-1})/(b_i - a_i c'_{i-1})$, then back-substitution $T_i = d'_i - c'_i T_{i+1}$). The two boundaries enter as:

- **Surface (radiative).** The skin temperature T_s is found each step by Newton iteration on the non-linear energy balance $R(T_s) = (1-A)S - \varepsilon\sigma T_s^4 - K_s\,(T_s - T_0)/(\tfrac{1}{2}\Delta z_0) = 0$, with $R'(T_s) = -4\varepsilon\sigma T_s^3 - 2K_s/\Delta z_0$; the converged T_s feeds the cell-0 ghost terms.
- **Bottom (geothermal).** A Neumann flux: b_N drops its right-face term and the source d_N += $\Delta t\, Q_b/c_N$ is added.

(Code: `lunar/solver.py`, functions `_step`, `_thomas`, `_solve_surface_newton`.)

2.3 The spin-up loop

The driver repeats: run all ~709 steps of one lunation, compare the new cycle to the previous one, and stop when $\max_i |T_i^{(k+1)} - T_i^{(k)}| < 0.01$ K. The *only* knobs are the starting profile and the number of lunations.

2.4 Why 30 lunations fails

Fig. 1 (animated in `spinup.gif`) shows it. The Sun's daily wave penetrates only ~10–20 cm; below that, what changes the deep regolith is the slow downward conduction of the *mean*, advancing $\sim z^2$ in time. After 30 lunations only the top ~0.5 m has adjusted; the regolith from 1–5 m still sits at the initial guess, and the steady slope (Eq. (3)) has not formed. The sensors at 0.8–2.4 m lie exactly in that unsettled zone — hence the bias.

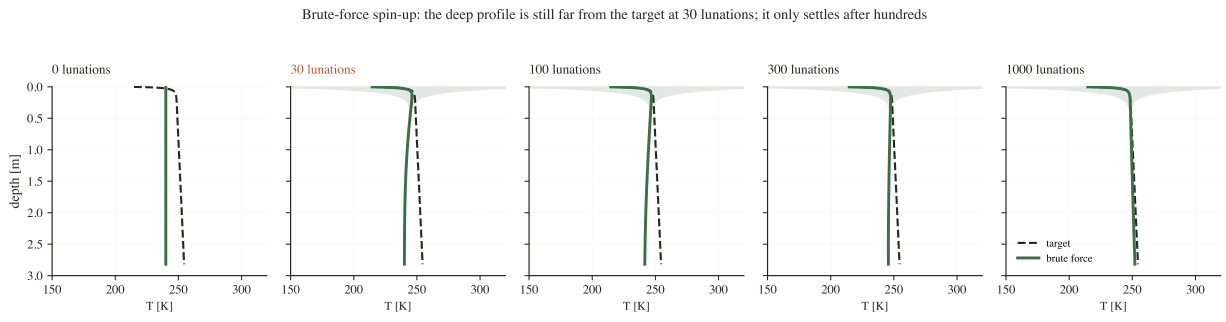

Figure 1: Old method (brute force). Green = current profile; dashed = the true steady state; shaded = daily swing. At 30 lunations only the top has moved; it takes hundreds of lunations to reach the target.

3 The new calculation, in full detail

The new solver keeps the *exact same* time-stepper of §2.2 — it does not change the physics. It only changes *how long* that stepper runs and what happens between runs. It alternates two moves (Fig. 2; animated in `newmethod.gif`).

3.1 Step A — settle the skin

Run the stepper for a *short* block of n lunations from the current profile. Because the skin is shallow it equilibrates in a handful of lunations. Read the cycle-mean "anchor" temperature $\langle T\rangle(z_0)$ just below the skin.

3.2 Step B — reconstruct the deep profile

From the anchor, integrate Eq. (3) *downward* to set every deeper cell. The code uses a midpoint (RK2) step (`_reconstruct_subskin`):

$$q_i = Q_b - u_i, \quad T_{1/2} = T_i + \tfrac{1}{2}\frac{q_i}{K(T_i, z_i)}\Delta z, \quad T_{i+1} = T_i + \frac{q_i}{K(T_{1/2}, z_{1/2})}\,\Delta z. \tag{6}$$

This is the move people find surprising: the deep temperatures are not simulated, they are *computed* by walking down a known slope. With $Q_b = 21\,\mathrm{mW\,m^{-2}}$ and $K \approx 9\,\mathrm{mW\,m^{-1}K^{-1}}$ at depth, the slope is $\sim$2.3 K/m, so each metre adds $\sim$2.3 K from the anchor downward (animated in `reconstruct.gif`).

3.3 The rectification correction u_i

Near the skin the daily wave is still large, and the cycle-mean of the *nonlinear* flux is not quite $K(\langle T\rangle)\,\partial_z\langle T\rangle$; the difference is the "rectified" eddy term $u(z) = \langle K\,\partial_z T\rangle - K(\langle T\rangle)\,\partial_z\langle T\rangle$, computed exactly from the cycle (`_rectified_flux`) and subtracted in Eq. (6). It is a few percent of Q_b at the anchor and negligible below $\sim$1 m, but including it keeps the closure exact.

3.4 The outer iteration (exact schedule)

Steps A–B are iterated as a two-stage schedule (`solve_periodic_equilibrium`):

Stage	anchor z_0	inner lunations n	max outer its	stop when drift <
1 (coarse)	0.25 m	4	4	0.10 K
2 (fine)	0.55 m	12	20	0.005 K

Stage 1 cheaply kills the bulk of the initial-guess error high in the column; stage 2 anchors below the rectification zone and iterates to tolerance. "Drift" is the change in the anchor temperature between successive outer iterations. In practice this converges in about a dozen outer rounds — **$\sim$150 lunations of stepping in total** (the rest of each iteration is the cheap algebra of Eq. (6)).

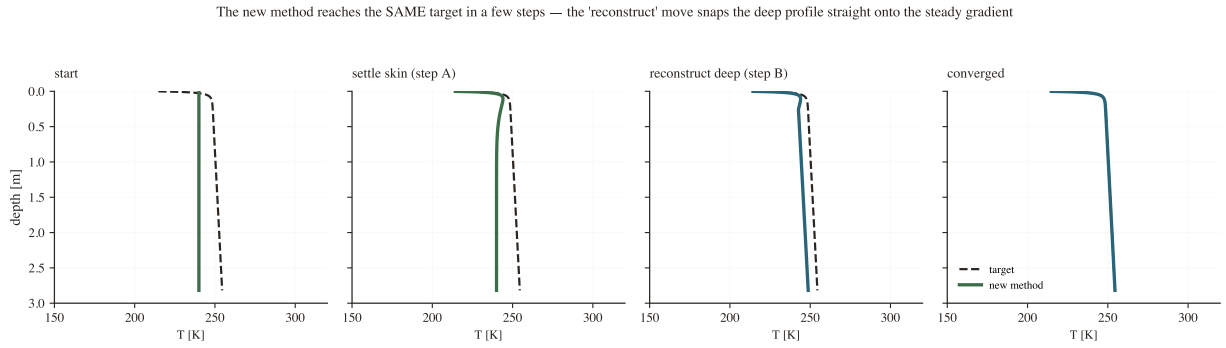

Figure 2: New method. A short spin-up settles the skin (Step A); the reconstruct move snaps the deep profile onto the steady gradient (Step B); a dozen short rounds refine it to the target.

4 The precise difference between the two calculations

Both run the identical Crank–Nicolson stepper. The difference is entirely in how the *deep* profile is obtained:

	Old (brute force)	**New (flux-anchored)**
How the deep profile is set	by waiting for heat to diffuse down over the full $\tau \sim 10^3$ lunations	computed directly from $d\langle T\rangle/dz = Q_b/K$ (Eq. (6)) after a short skin spin-up
Stepping done	~1000+ lunations in one long run	~150 lunations, in ~12 short blocks
Depends on the initial guess?	yes (if stopped early)	no — the deep profile is reconstructed each round
Convergence test	cycle-to-cycle ΔT (can give a false positive)	anchor drift + flux closure on Q_b (cannot)
Wall time per solve	~4 min	~9 s

In one sentence: the old method *discovers* the deep gradient Q_b/K slowly by letting heat diffuse; the new method *writes it down* from energy conservation and only simulates the fast shallow part. Same physics, same stepper, same answer — ~30× less work.

5 How we know the new method is right

Five independent checks, in increasing strength:

(1) Uniqueness (the argument). Given the anchor value, Eq. (3) is a first-order ODE with a smooth right-hand side, so it has exactly one solution below the anchor; the anchor itself is fixed by the fast surface balance. The periodic steady state is therefore unique. Any method that satisfies both defining conditions (periodic skin + flux closure) has found it — and both methods do.

(2) Self-certification. Every solve returns two diagnostics: the anchor *drift* between outer iterations (we require < 0.03 K; typically ~3 mK) and the *flux closure* $\max|\langle q\rangle - Q_b|/Q_b$ below the anchor (we require $< 2\%$; Fig. 5). These are exactly the test the old fixed spin-up could not pass.

(3) Guess-independence. Starting the new solver from 240 K and from 260 K gives sensor-depth temperatures agreeing to < 0.03 K. The very symptom that exposed F1 is now gone.

(4) Brute-force cross-validation (the decisive one). We take the *old* method and simply run it far longer. As the lunations increase its profile marches monotonically onto the new method's answer (Fig. 3), and from two different starts it lands on the same curve to tens of mK at 3000 lunations (Fig. 4). The two methods agree where it can be checked directly.

(5) Drift/rectification validation. A 120-lunation drift test confirms the neglected higher-order term is $< 1\%$ of Q_b below the anchor, so Eq. (6) closes to the stated tolerance. (Tests: `tests/test_equilibrium.py`.)

6 Porting to C++

6.1 Where the time goes

A solve spends essentially all its time in the inner step (§2.2), run hundreds of thousands of times. In the current Python, the tridiagonal *solve* (`_thomas`) is already JIT-compiled with Numba, but the matrix *assembly* in `_step` (building a, b, c, d and the face means each step) is interpreted Python over ~69-element arrays — many cheap operations per step, which is exactly the pattern where interpreter overhead dominates. That is the part a compiled language removes.

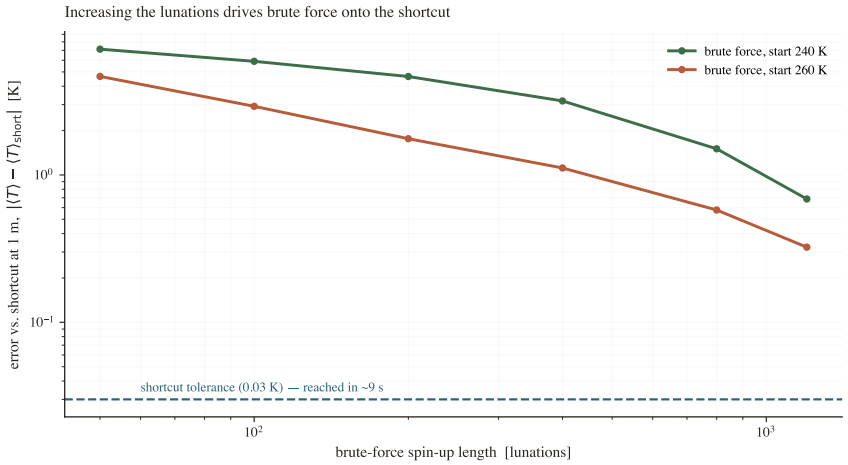

Figure 3: Check (4): brute-force error vs. the shortcut falls steadily as the spin-up lengthens; two starts converge to the same place.

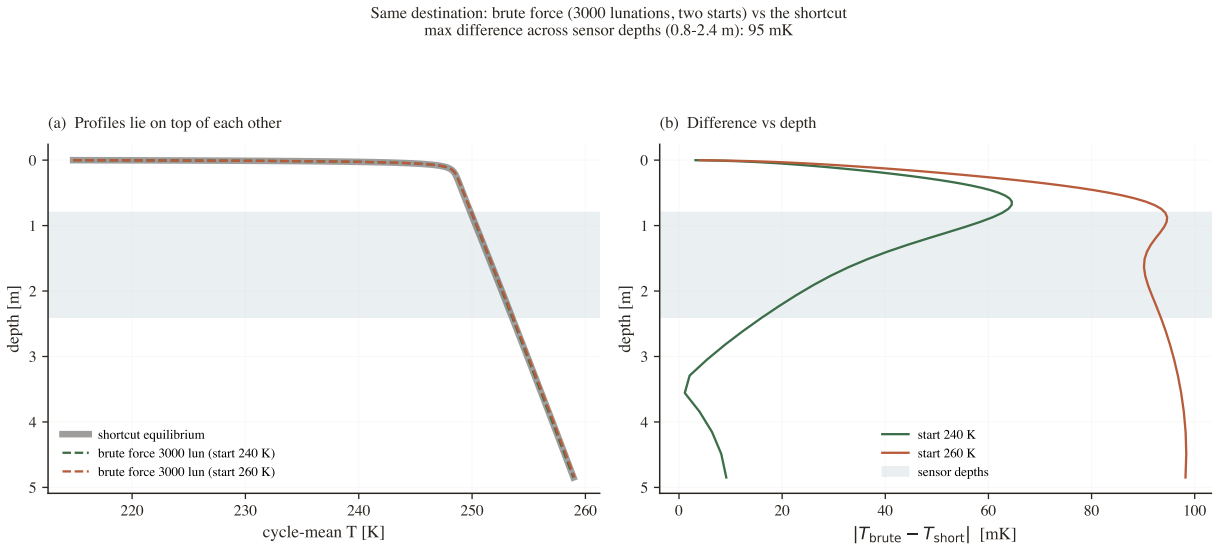

Figure 4: Brute force to 3000 lunations from 240 K and 260 K (dashed) on the shortcut (solid): indistinguishable; the residual is brute force still finishing.

6.2 What to port, what to keep

The numerics are already isolated in three small, I/O-free modules — port these:

- `grid.py` — build the geometric mesh (trivial array construction);
- `solver.py` — the per-step assembly, the harmonic face means, the Thomas solve, and the Newton surface balance (§2.2);
- `equilibrium.py` — the outer A–B loop and the RK2 reconstruction (Eq. (6)).

Keep `config.py` (emit it as JSON so the C++ side reads the same parameters), and leave the retrieval, statistics, and all plotting in Python calling the fast core through a thin binding (e.g. pybind11). The three-layer code layout (Fig. 6) is built for exactly this split.

6.3 Expected gain, honestly

For this tight small-array loop a compiled core typically buys ~10–100× on the stepping. **But** much of that is reachable *without* C++: Numba already compiles `_thomas`, and applying the same `@njit` (or a NumPy

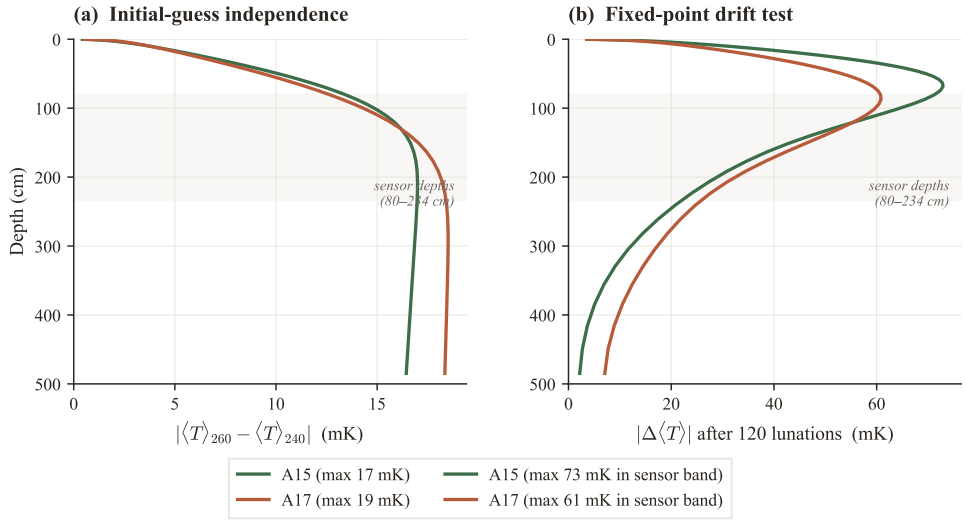

Figure 5: Check (2): guess-independence and mean-flux closure — the returned proof that a solve reached the steady state.

vectorisation) to `_step` would capture most of the gain in pure Python. So C++ is the ceiling, not the only route. The three speed-ups are independent and multiply: the algorithm (the shortcut, ~30×, already done) × compilation (C++/Numba) × parallelism across independent solves (the K_d sweep fans over CPU cores). For the present paper, Python plus the shortcut is already fast enough; C++ pays off when scaling to many DEM pixels or a mission pipeline that runs the solver millions of times.

6.4 How to validate the port

Port bottom-up and regression-test each layer against the Python reference: the grid arrays must match bit-for-bit; the stepper must reproduce the analytic constant-coefficient thermal wave (already used in `tests/test_solver.py`); and the full solver must reproduce the committed steady-state profiles and the `drift/flux_closure` diagnostics. The existing 37-test suite is the cross-language regression harness.

How to defend each claim in one line

- *Why a new method?* A fixed spin-up doesn't reach steady state at sensor depth, so the initial guess biased K_d (F1).
- *What it does differently?* It imposes the deep gradient Q_b/K directly instead of waiting ~10^3 lunations for diffusion to find it.
- *How do we know it's right?* It is unique by construction, it certifies its own convergence, it is guess-independent, and brute force run long converges onto it.
- *Faster?* ~9 s vs ~4 min per solve; portable to C++ for another ~10–100× at scale.

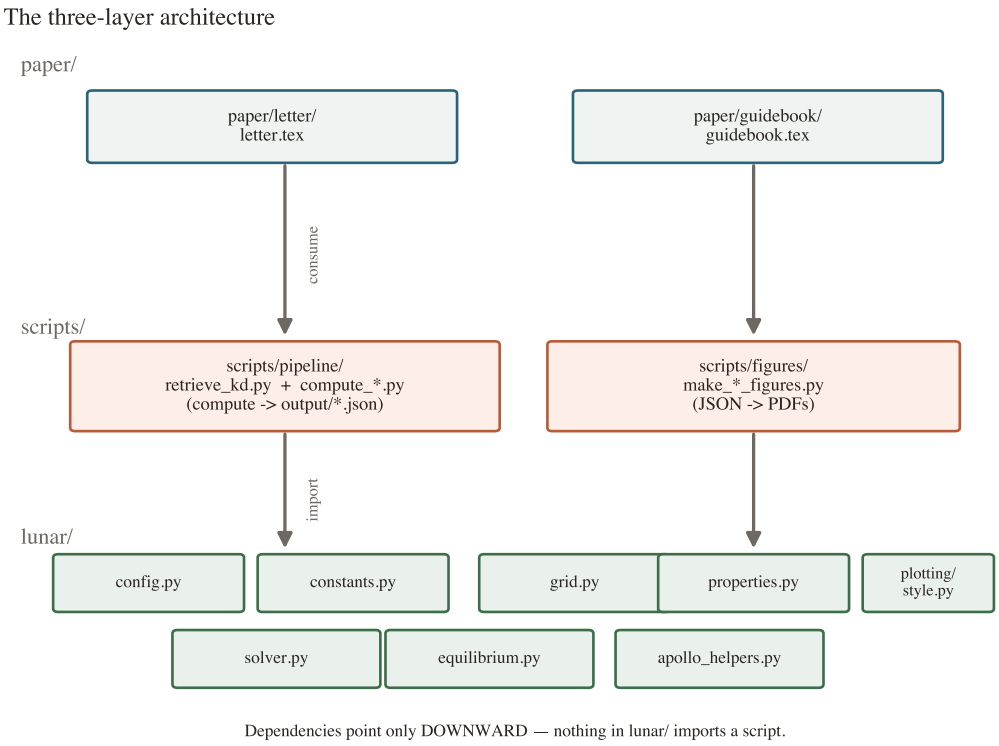

Figure 6: The codebase is layered so the numerics (`lunar/`) can be replaced by a C++ core while the drivers and analysis stay in Python.